Por que Reservoir Computing funciona: memoria, nao linearidade e capacidade de representacao

Resumo. *Este artigo responde a pergunta que ficou aberta depois do primeiro pipeline: por que o desempenho do RC muda tanto quando ajustamos seus hiperparametros? Para responder, conectamos memoria, nao linearidade e capacidade de representacao a resultados computacionais reais do projeto. Comecamos por uma leitura teorica do reservatorio, mostramos como o* `leak_rate` *controla a escala temporal efetiva, como o* `spectral_radius` *influencia estabilidade e mistura e como o tamanho do reservatorio afeta a base de representacao. Em seguida, analisamos os resultados de varreduras reais sobre* `n_reservoir`*,* `spectral_radius` *e* `leak_rate`*. O artigo nao apenas explica RC em abstrato: ele mostra o que essas ideias produzem, ou destroem, no recorte Favorita.*

0.1. 1. O que o leitor vai aprender

Ao final deste artigo, voce sera capaz de:

1. explicar memoria e nao linearidade no RC em termos matematicos;
2. entender por que `spectral_radius` e `leak_rate` mudam o comportamento do modelo;
3. interpretar tabelas de tuning do RC;
4. diferenciar capacidade de representacao de mera quantidade de unidades;
5. decidir o que vale testar antes de partir para QRC.

0.2. 2. Memoria: o passado precisa continuar presente

Em RC, a memoria nao aparece como um vetor de lags explicitamente escrito. Ela aparece no proprio estado do reservatorio. Se linearizarmos a dinamica em torno de uma regiao pequena, obtemos aproximadamente

$$x_t \approx Ax_{t-1} + \alpha W_{in} u_t + \alpha b, \qquad A = (1-\alpha)I + \alpha W_{res}.$$

Desdobrando a recursao:

$$x_t \approx \sum_{k=0}^{\infty} A^k (\alpha W_{in} u_{t-k} + \alpha b).$$

Essa expressao mostra a intuicao correta: o estado atual e uma superposicao ponderada do passado. A velocidade com que o passado "desaparece"depende da matriz efetiva A, logo depende de `spectral_radius` e `leak_rate`.

0.2.1. 2.1 O papel do leak rate

O `leak_rate` regula quanto do estado novo substitui o estado antigo. Valores pequenos alongam a memoria; valores grandes tornam a resposta mais reativa.

No projeto, o `leak_rate` default foi `0.15`, mas a varredura mostrou que `0.30` melhora levemente o resultado neste recorte.

leak_rate	MAE	RMSE	WAPE	sMAPE
0.05	388.180	491.504	0.1793	0.1982
0.1	357.675	474.461	0.1652	0.1822
0.15	324.294	423.417	0.1498	0.1653
0.3	317.767	419.947	0.1467	0.1621
0.5	770.680	1035.544	0.3559	0.5702

Esse resultado ensina uma licao importante: uma memoria mais longa nao e automaticamente melhor. Se a serie responde fortemente a promocao e sazonalidade semanal, uma resposta um pouco mais rapida pode ajudar.

0.2.2. 2.2 O papel do washout

O `washout` separa o regime transiente inicial da dinamica util para treino:

$$\Phi = [\phi_w, \phi_{w+1}, \ldots, \phi_T]^\top.$$

Sem `washout`, o readout pode aprender artefatos de inicializacao em vez de aprender uma representacao estabilizada da serie.

0.3. 3. Nao linearidade: por que a serie nao cabe em uma regra unica

O reservatorio usa a nao linearidade `tanh`:

$$\widetilde{x}_t = \tanh(W_{res} x_{t-1} + W_{in} u_t + b).$$

Essa nao linearidade e o que permite ao modelo misturar efeitos de calendario, promocao e demanda previa sem impor uma relacao puramente aditiva ou puramente linear.

Em previsao de demanda, isso importa porque:

- a mesma promocao pode gerar efeitos diferentes em dias diferentes;
- o mesmo dia da semana pode ter resposta diferente dependendo do historico imediato;
- a relacao entre passado e presente nao precisa ser monotona nem linear.

0.4. 4. Capacidade de representacao: tamanho importa, mas nao sozinho

Um erro comum de iniciantes e supor que "mais unidades"sempre significa "mais desempenho". A varredura de `n_reservoir` no projeto mostra o contrario.

n_reservoir	MAE	RMSE	WAPE	sMAPE
40	1278.623	1721.270	0.5904	0.4393
80	324.294	423.417	0.1498	0.1653
120	394.030	464.884	0.1820	0.2026
160	430.945	565.166	0.1990	0.2359

A configuracao `n_reservoir = 80` foi a melhor entre as testadas. Quando o reservatorio ficou pequeno demais (`40`), o erro explodiu. Mas aumentar para `120` e `160` tambem piorou.

Esse comportamento sugere que capacidade util e compatibilidade com o problema importam mais do que simplesmente aumentar a dimensionalidade.

0.5. 5. Estabilidade e mistura: o papel do spectral radius

O `spectral_radius` reescala a matriz recorrente para controlar o quanto a dinamica interna amplifica ou dissipa o estado.

spectral_radius	MAE	RMSE	WAPE	sMAPE
0.2	275.280	347.305	0.1271	0.1420
0.4	340.569	454.497	0.1573	0.1764
0.6	324.294	423.417	0.1498	0.1653
0.8	871.792	1188.448	0.4026	0.6992
1.0	500.267	632.914	0.2310	0.2519

A varredura mostra tres regimes didaticamente importantes:

1. `0.2` produziu o melhor conjunto de metricas entre os valores testados;
2. `0.6` ficou em uma faixa intermediaria e foi o default do primeiro pipeline;
3. `0.8` e `1.0` degradaram muito a estabilidade, com explosao de erro.

Em outras palavras, este recorte prefere um reservatorio mais contido do que o default inicial sugeria.

0.6. 6. Como ler os hiperparametros do RC no codigo

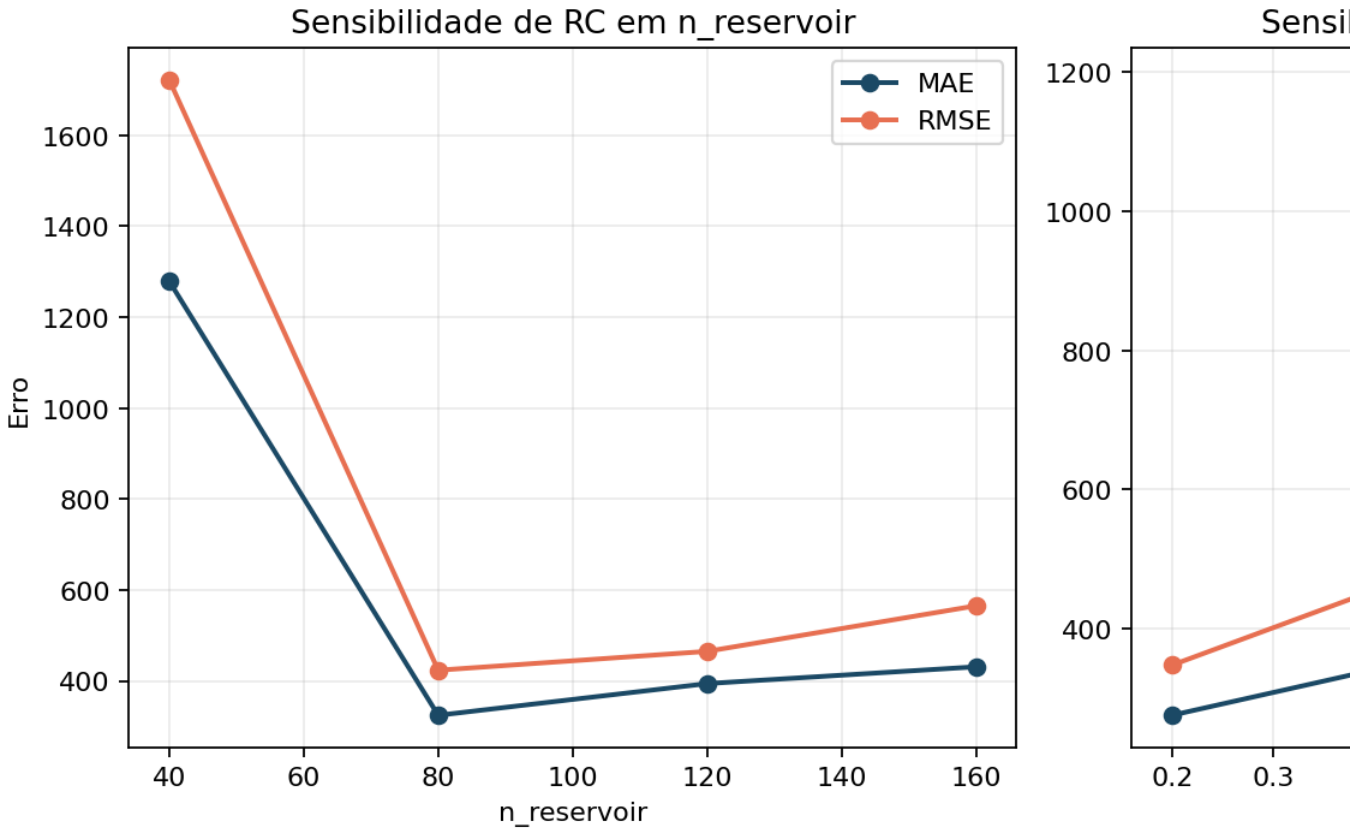
O ponto de entrada do tuning esta em `RCConfig`:

```
@dataclass(frozen=True)
class RCConfig:
    n_reservoir: int = 80
    spectral_radius: float = 0.6
    leak_rate: float = 0.15
```

```
input_scale: float = 0.8
bias_scale: float = 0.1
washout: int = 14
ridge_alpha: float = 1e-3
seed: int = 7
```

Um modo pratico de estudar o modelo e variar apenas um hiperparametro por vez, mantendo os demais fixos. Foi exatamente isso que os artefatos deste artigo fizeram.

0.7. 7. O que o resultado atual do RC sugere



O resumo visual permite uma leitura didatica direta:

- o modelo e sensivel ao `spectral_radius`;
- o `leak_rate` possui um ponto intermediario util;
- o tamanho do reservatorio tem um regime otimo local, e nao monotonia.

Isso confirma uma licao central para a serie: o RC nao deve ser avaliado apenas por uma configuracao arbitraria. Seu desempenho depende de como ajustamos memoria, mistura e capacidade.

0.8. 8. O que RC faz bem e o que ainda nao faz bem aqui

0.8.1. 8.1 O que RC faz bem

- construi uma representacao temporal dinamica de baixo custo de treinamento;
- absorve sinais de promocao e calendario sem uma engenharia tabular extensa;
- oferece uma ponte conceitual excelente para o QRC.

0.8.2. 8.2 O que RC ainda nao faz bem aqui

- ainda nao supera os melhores modelos classicos neste recorte;
- ainda depende de tuning cuidadoso para evitar instabilidade ou sub-representacao;
- ainda nao incorpora a riqueza completa do dataset Favorita.

0.9. 9. Passo-a-passo para reproduzir o estudo de sensibilidade

Um leitor que queira repetir este artigo pode seguir esta sequencia:

1. executar `python code/rc/run.py` para fixar a configuracao base;
2. alterar `RCConfig` em `code/rc/model.py` ou criar instancias novas em um script de experimento;
3. rerodar a previsao para cada combinacao;
4. comparar `MAE`, `RMSE`, `WAPE` e `sMAPE`;
5. guardar o resultado em CSV e imagem, como foi feito em `computational_results_20260402_222902/`.

Esse processo prepara o leitor para a fase seguinte da serie, em que a comparacao deixa de ser apenas "baseline versus RC"e vira um benchmark mais amplo.

0.10. 10. Conclusao

Este artigo mostrou que o RC funciona porque oferece memoria distribuida e nao linearidade controlada, mas que essa vantagem depende de como configuramos o sistema dinamico. O proprio comportamento das varreduras do projeto ensina essa licao melhor do que qualquer definicao abstrata: parametros inadequados destroem o desempenho, enquanto escolhas razoaveis tornam o reservatorio competitivo o suficiente para merecer comparacao seria.

0.11. Entregaveis associados no repositorio

- implementacao do RC: `code/rc/model.py`
- artefatos de sensibilidade: `computational_results_20260402_222902/`
- arquivos principais: `rc_sweep_n_reservoir.csv`, `rc_sweep_spectral_radius.csv`, `rc_sweep_leak_rate.csv`, `rc_sensitivity_summary.png`

0.12. Referencias

- Jaeger, H. The "echo state"approach to analysing and training recurrent neural networks.
- Lukosevicius, M. A practical guide to applying echo state networks.
- Verstraeten, D. et al. Memory versus non-linearity in reservoirs.